

ASPERITAS AI AGENT DEVELOPMENT CONTROL PLANE

GPT 답변 성능 업그레이드 및 AI Agent 개발 과정 업그레이드 소스 첨부 문서

Version v1.0 | Generated 2026-07-04 KST | Use: ChatGPT Project Source, Codex Prompt Source, RAG/Agent Development Doctrine

- 목적: 이 채팅에서 학습한 OpenAI, Anthropic, LangGraph, MCP, eval, tracing, observability, Codex, RAG/agent 프레임워크 원칙을 하나의 실행 문서로 통합한다.
- 사용처: ChatGPT 프로젝트 소스 첨부, Codex 프롬프트 생성, AGENTS.md/README 업그레이드, PR 품질 게이트, RAG/eval/observability 설계.
- 경계: 이 문서는 운영 헌법과 외부 벤치마크 요약이며, 실제 repo에 vector DB, KG, eval suite, legal review, production tracing이 이미 구현되었다는 증거가 아니다.

Executive Bottom Line

Asperitas AI Agent의 목표는 generic chatbot이 아니다. 목표는 biodiversity access를 structured biological data, source-grounded intelligence, DBTL validation, IP, compliance trust, and scalable biological infrastructure로 전환하는 operating system이다. 따라서 답변 성능과 개발 과정은 모델 능력보다 더 넓은 control plane - evidence, state, tool contracts, guardrails, tracing, evals, review gates - 로 관리해야 한다.

0. 사용 방식과 Source Governance

이 문서는 ChatGPT 프로젝트 소스에 첨부해 “답변 품질”과 “에이전트 개발 과정”을 동시에 강화하기 위한 기준 문서다. Codex에게 그대로 붙여넣는 단일 프롬프트가 아니라, 모든 프롬프트와 PR 작업에 적용하는 상위 operating doctrine이다.

- P0 내부 헌법: Asperitas AOS/PRIME/AGENTS/Ultimate Prompt 계열. 내부 방향성과 우선순위를 정한다.
- P1 공식 기술 문서: OpenAI, Anthropic, LangChain/LangGraph, MCP 등 primary source. 구현 의사결정의 근거다.
- P2 평가/관측/프레임워크 문서: Ragas, DeepEval, Phoenix, Langfuse, DSPy, CrewAI, AutoGen, Semantic Kernel, Google ADK, Vercel AI SDK 등. 비교와 선택 규칙에 사용한다.
- P6 외부 벤치마크: 직접 구현 사실이 아니라, Asperitas agent 개발 품질을 높이는 operating benchmark로만 취급한다.

Non-negotiable: Source-grounded, auditable, compliance-aware, token-efficient, MVP-gated. 답변이 좋아 보이는 것보다 근거, 재현성, 안전성, 평가 가능성이 우선이다.

1. 이 채팅에서 통합 학습한 외부 원칙

Source Group	학습 핵심	Asperitas 적용
OpenAI API / Business	Agent Builder, Agents SDK, grounding tools, evals, security/admin controls, enterprise ROI	Agent engineering + enterprise governance + business value gate를 기본 프롬프트 구조에 삽입
OpenAI Release Notes / Academy	최신 기능 변화 감지, Goal mode, memory governance, workflow 재사용, AI fluency	PR 시작 전 release-note-aware gate, 반복 가능한 workflow capture, review point 설계
OpenAI Agents SDK	Responses API vs Agents SDK 구분, one specialist contract, tools, handoffs, state, approvals	API/SDK decision gate와 specialist contract gate 추가
OpenAI Tracing	LLM generation, tool call, guardrail, handoff, custom event를 trace/span으로 관측	RAG/agent PR에 workflow_name, group_id, metadata, span coverage, sensitive data policy 요구
OpenAI Guardrails	input/output/tool guardrails와 tripwire	biosecurity, IP, CITES/license, DB write, external output 전에 blocking guardrail 설계
OpenAI Evals / Graders	BDD식 eval, dataset, grader, pass threshold, regression loop	구현 전 eval objective, fixtures, grader, failure taxonomy, skipped-eval rationale 요구
OpenAI Codex Loop	prompt, tools, sandbox, AGENTS.md, skills, environment context, agent loop	Codex prompt에 cwd/shell/permissions/AGENTS.md/tests/evals/report 구조 반영
Anthropic Effective Agents	simple-first, workflows vs agents, prompt chaining/routing/parallel/orchestrator/evaluator/autonomous patterns	architecture ladder와 complexity justification gate 고정
Anthropic Biology Agents	생물학 agent 병목은 모델 지능보다 데이터 인프라 비표준성. deterministic retrieval layer가 정확도 상승	biology/RAG에서는 LLM 추측 금지. gene ID, schema, retrieval, evidence, metadata를 deterministic layer로 처리
LangGraph / LangSmith	stateful graph orchestration, checkpointers, stores, HITL, time travel, observability	LangGraph는 state/checkpoint/HITL/fault tolerance가 필요할 때만 사용
MCP	AI application과 external systems/tools/workflows를 연결하는 open standard	MCP connector마다 auth, permission scope, schema validation, side-effect boundary, audit log, injection review 요구
Ragas / DeepEval	RAG metrics, tool call accuracy, agent goal accuracy, faithfulness, contextual precision/recall	RAG answer quality와 agent trajectory quality를 별도 metric으로 분리
Phoenix / Langfuse / LangSmith	trace, span, eval, prompt experiments, cost/token/latency monitoring	observability stack 선택 기준과 trace-eval 연동 구조 반영
DSPy / Vercel AI SDK / ADK / CrewAI / AutoGen / SK / LlamaIndex	framework-specific strengths	framework selection matrix와 “default 금지, 필요성 증명 후 선택” 규칙 추가

2. GPT 답변 성능 업그레이드 원칙

답변 품질은 “모델이 더 똑똑하게 생각하는 것”만으로 올라가지 않는다. 좋은 답변은 목표, evidence contract, constraints, output contract, verification, stop rules가 결합될 때 안정적으로 재현된다.

1. Outcome-first: 먼저 목표, 성공 기준, 출력 형식, 금지 범위, 검증 기준을 고정한다.
2. Evidence-first: source ID, citation key, evidence span, registry version, compliance tag를 보존한다.
3. Compact but complete: 장황함은 제거하되, 근거/제약/검증/중단 규칙은 절대 줄이지 않는다.
4. Model routing: 복잡한 계획/검토/위험 판단은 reasoning model, 명확한 실행/변환/반복 작업은 빠른 모델/도구를 쓴다.
5. Refusal and uncertainty: 근거가 부족하면 unsupported, uncertain, needs-human-review로 분류하고 과잉 확신을 금지한다.
6. Reusable output: 답변은 다음 PR, eval, template, AGENTS.md rule, source registry rule로 재사용 가능한 형태를 남긴다.

2.1 기본 답변 프레임

Goal:
사용자의 실제 의사결정 또는 작업 산출물을 먼저 정의한다.

Context:
필요한 프로젝트/소스/버전/제약만 넣는다. 불필요한 컨텍스트는 줄인다.

Evidence Rules:
근거가 필요한 주장에는 source ID, citation key, evidence span, registry/status를 보존한다.

Constraints:
compliance, privacy, biosafety, IP, token/cost/latency, negative scope를 명시한다.

Output Contract:
표, 체크리스트, JSON, PR report, Codex prompt 등 사용 가능한 형식으로 고정한다.

Verification:
어떤 test/eval/trace/human review로 맞는지 확인할지 적는다.

Stop Rules:
근거 부족, 권한 부족, 위험 작업, blocking gap에서는 멈추고 보고한다.

2.2 답변 품질 점검표

Axis	Check
정확성	출처와 일치하는가? unsupported claim이 없는가?
근거성	citation, evidence span, source registry/status가 보존되는가?
실행성	다음 행동, 파일, 테스트, PR 기준으로 바로 쓸 수 있는가?
안전성	bio/IP/compliance/confidentiality 위험을 분류했는가?
재사용성	template, eval case, AGENTS.md rule, regression test로 전환 가능한가?
효율성	필요한 정보만 사용했는가? latency/cost/token 낭비가 없는가?

3. Agent Development Control Plane

Agent 개발의 핵심은 “더 많은 agent를 붙이는 것”이 아니라, 작업 복잡도에 맞는 최소 구조를 고르고, 그 구조에 상태, 도구, guardrail, trace, eval, human review를 연결하는 것이다.

3.1 Architecture Ladder

1. deterministic code/helper
2. single LLM/RAG/tool call
3. fixed workflow
4. stateful workflow
5. agent
6. multi-agent / graph framework

- 단순 변환, schema formatting, deterministic validation은 code/helper로 끝낸다.
- 단일 질문, 작은 RAG answer, claim helper는 single LLM/RAG/tool call이 우선이다.
- 단계가 예측 가능하면 workflow다. 예측 불가능하고 model이 tool/step을 선택해야 agent다.
- multi-agent는 specialist separation이 reliability, evaluation, control을 실제로 개선할 때만 쓴다.
- LangGraph, Agents SDK, CrewAI, AutoGen 등은 complexity justification이 있어야 한다.

3.2 Complexity Justification Gate

Before adding complexity, answer:

- Why do simpler patterns fail?
- What quality/reliability/control gain is expected?
- What latency, cost, security, privacy, and maintenance risk is added?
- What eval evidence will prove the gain?
- What rollback path exists if the framework/agent path fails?
- Which fields, traces, guardrails, and datasets must be preserved?

4. Framework Selection Matrix

Tool/Framework	Use when	Avoid when
Plain Python / deterministic tools	bounded transformation, schema checks, citation map, metadata validation	stateful orchestration, open-ended tool choice, HITL resume
Responses API / single LLM call	one model call + tools + app-owned logic	app/server must own orchestration, approval, persistent state
OpenAI Agents SDK	tool execution, approvals, state, handoffs, specialist collaboration, MCP/runtime control	simple RAG answer or deterministic helper
LangGraph	long-running stateful graph, checkpoint/resume, HITL interrupt, branching, time travel, retries, streaming	short deterministic workflow with no state complexity
CrewAI	role-based crews/flows and procedural business workflows	single specialist or deterministic pipeline
AutoGen	multi-agent conversation/team protocols and experimental interaction patterns	production RAG without conversation-team complexity
Semantic Kernel	enterprise agent abstraction, planners, plugins/skills, Microsoft ecosystem	small Python-only helper or low-overhead RAG
LlamaIndex agents	RAG/data-agent loops, query engine tools, source/data-centric workflows	generic non-data workflow
Google ADK	enterprise-scale build/debug/deploy/evaluate/scale and production multi-agent deployment	local MVP without enterprise deployment needs
Vercel AI SDK	TypeScript-first web agents, tool calling, streaming UX, deployment	Python backend-only RAG with no web UX
DSPy	metric-driven prompt/program optimization after stable datasets and metrics exist	no dataset, no metric, early exploration
Phoenix / Langfuse / LangSmith	tracing, eval, prompt experiments, cost/token monitoring	never as replacement for domain evals or source contracts
Ragas / DeepEval / OpenAI Evals	RAG/agent metric suites, regression gates, grader orchestration	vibe-based manual review only

5. OpenAI Agents SDK Gate

Agents SDK는 기본값이 아니다. 먼저 Responses API 또는 plain workflow로 충분한지 판단한다. SDK는 app/server가 orchestration, tool execution, approvals, state, handoffs를 소유해야 할 때 사용한다.

Agents SDK Gate:

- Use Responses API/plain workflow if one model call + tools + app-owned logic is enough.
- Use Agents SDK only if the application must own orchestration, tool execution, approvals, persistent state, specialist collaboration, custom storage, MCP/tool runtime, or product-infrastructure integration.

If Agents SDK is justified, define:

1. Specialist contract: role, instructions, model/provider, tools, input/output schema.
2. Runtime/state: run loop, resumable state, final output, next-turn surface, persistence.
3. Tools/MCP: arguments, side effects, permissions, auth, schemas, timeout/retry.
4. Orchestration: handoff rules, final-answer owner, specialist boundaries.
5. Guardrails/HITL: validation, approval gates, fallback/refusal behavior.
6. Observability: traces, run metadata, tool logs, state transitions, sensitive-data handling.
7. Eval: dataset, graders, pass threshold, regression cases.
8. Production: latency, cost, security, storage, deployment risk.

6. LangGraph Architecture Gate

LangGraph는 durable execution, state, checkpointing, HITL, branching, fault tolerance, time-travel debugging이 필요한 workflow에 적합하다. 단순 RAG helper에는 과하다.

LangGraph Gate:
Use LangGraph only if workflow/state complexity justifies it.

Define before implementation:

- graph state schema
- nodes and edges
- conditional routing
- checkpoint/store design
- thread_id strategy
- interrupt/HITL approval points
- retries, timeouts, fallback/error handlers
- streaming requirements
- trace/eval metadata
- sensitive-data redaction
- regression coverage for routing, resume, HITL, citation faithfulness, compliance preservation

6.1 Asperitas RAG Graph Node Map

```
query_input
→ retrieval
→ reranking
→ evidence_compression
→ claim_extraction
→ claim_verification
→ citation_assembly
→ compliance_guardrail
→ human_review_if_high_risk
→ answer_metadata
→ eval_result_capture
```

7. Asperitas RAG/Bio State and Provenance Contract

생물학/RAG workflow에서는 모델이 source status, citation support, license/compliance, missing evidence를 추측하면 안 된다. 아래 필드는 retrieval, verification, answer generation, trace, eval 전 구간에서 보존되어야 한다.

Required State Fields:

```
query
normalized_query
source_registry_version
retrieval_candidates
reranked_chunks
chunk_ids
evidence_spans
claim_ids
support_status
citation_map
compliance_tags
schema_version
prompt_version
model_route
trace_id
eval_profile
residual_uncertainty
failure_modes
diagnostics
```

7.1 Biology Hard Rule

- gene IDs, organism/taxonomy identifiers, sequence coordinates, database schemas, metadata conventions, source licensing, CITES/Nagoya/LMO/compliance status는 deterministic tool/schema layer가 우선이다.
- LLM은 hypothesis generation, synthesis, explanation, reviewer 역할에는 유용하지만, source truth와 compliance truth를 “추측”하는 구성은 금지한다.
- high-risk bio/IP/compliance answer는 external output 또는 persistent write 전에 human approval을 통과해야 한다.

8. Tool / MCP / ACI Contract

Tool 성능은 prompt 성능만큼 중요하다. MCP/tool connector는 agent에게 외부 능력을 주는 동시에 data exfiltration, prompt injection, side effect risk를 만든다. 따라서 tool마다 contract가 필요하다.

```
Tool / MCP Contract:
name:
purpose:
exact_input_schema:
exact_output_schema:
examples:
edge_cases:
failure_modes:
side_effects:
auth_permission_scope:
timeout_retry_policy:
audit_logging:
idempotency_key:
dry_run_mode:
prompt_injection_review:
data_exfiltration_review:
sensitive_data_policy:
```

- Mistake-resistant design: absolute paths, explicit IDs over names, enum fields, dry-run before writes, structured outputs, schema validation.
- Side-effecting tools: DB writes, ingestion, PR merge, external messages, production config changes는 human approval 전에는 실행하지 않는다.
- MCP는 표준 연결 방식으로 유용하지만, auth scope, permission boundary, audit log, guardrail이 없는 MCP는 생산 시스템에 연결하지 않는다.

9. Guardrails and Human-in-the-loop

Layer	Where	Detect	Tripwire Behavior
Input guardrail	user prompt, uploaded source, retrieved text	prompt injection, unsafe bio/security intent, scope violation, license ambiguity, excessive cost risk	block / ask human / safe-mode route
Tool guardrail	custom function tool before/after execution	bad args, missing auth, dangerous side effects, DB write, ingestion, external call	dry-run / require approval / abort
Output guardrail	final answer, JSON, PR report, citation map	unsupported claim, citation drift, schema invalidity, compliance leakage, confidentiality breach	refuse / mark uncertain / require review

```
Human Approval Required Before:
- unlicensed source ingestion
- external publication or customer-facing bio/compliance output
- DB write or source registry mutation
- schema migration
- PR merge or release tag
- confidential IP, contract, unpublished experiment, CITES/Nagoya/LMO sensitive decision
- costly long-running tool or background job
```

10. Tracing and Observability Gate

Tracing은 선택 로그가 아니라 audit/debug backbone이다. 고위험 RAG/agent PR은 trace boundaries, spans, metadata, sensitive-data policy를 명시해야 한다.

```

Tracing Gate:
workflow_name:
group_id_or_thread_id:
trace_id_strategy:
metadata:
  PR_version_ID
  prompt_version
  model_route
  source_registry_version
  schema_version
  eval_profile
  compliance_mode
  dataset_version
  experiment_tag
expected_spans:
  LLM_generation
  retrieval
  reranking
  evidence_compression
  claim_extraction
  claim_verification
  citation_assembly
  compliance_guardrail
  tool_calls
  handoffs
  human_approval
  retries_errors
  latency_cost
  eval_results
sensitive_data_policy:
  redact_or_disable_payload_capture_for_confidential_bio_IP_compliance_data
flush_policy:
  flush_traces_after_long_running_background_worker_context_if_immediate_export_needed

```

10.1 Observability Report

```

PR Observability Report:
- Trace coverage added/unchanged:
- workflow_name/group_id strategy:
- Spans covered:
- Sensitive-data handling:
- Debug/eval value:
- Latency/cost impact:
- Residual blind spots:

```

11. Evaluation and Regression Gate

Eval은 구현 후 느낌으로 확인하는 것이 아니라, 구현 전에 기대 행동을 정의하는 behavior-driven development다. Trace로 failure를 발견하고, 반복되는 failure는 dataset/eval/regression test로 승격한다.

```

Eval Design Gate:
objective:
expected_behavior:
dataset_or_fixtures:
  real_cases
  synthetic_cases
  domain_expert_cases
  production_log_cases
  edge_cases
  adversarial_cases
graders:
  deterministic_checks
  text_similarity
  model_grader
  python_grader
  human_domain_expert_review
pass_threshold:
failure_taxonomy:
regression_scope:
baseline_or_main_branch_comparison:
skipped_eval_rationale:

```

11.1 Asperitas RAG/Bio Eval Metrics

Area	Metrics
Retrieval	contextual recall, contextual precision, contextual relevancy, source coverage, chunk ID correctness
Generation	faithfulness, answer relevancy, citation faithfulness, evidence-span correctness, unsupported claim rate
Verification	claim verification accuracy, support status accuracy, uncertainty handling, refusal safety
Compliance	CITES/Nagoya/LMO/license tag preservation, sensitive data leakage, external output safety
Agent trajectory	tool call accuracy, tool argument correctness, handoff/routing accuracy, agent goal accuracy
Operational	schema validity, latency, cost, token usage, trace coverage, regression stability

11.2 Failure Taxonomy

Common Failure Classes:

- retrieval_miss
- retrieval_noise
- citation_drift
- evidence_span_mismatch
- unsupported_claim
- schema_only_pass
- fixture_overfitting
- compliance_tag_loss
- prompt_injection_followed
- tool_arg_error
- handoff_owner_confusion
- trace_gap
- high_latency_or_cost
- unsafe_refusal_or_non_refusal

12. Multi-agent and Competitive Verification

Multi-agent의 가치는 여러 역할 이름을 붙이는 것이 아니라, 서로 다른 실패 탐지 압력을 만드는 것이다. 구현자와 검증자의 목표를 분리하고, red-team과 compliance reviewer를 독립시켜야 한다.

Recommended Role Split:

Planner:
goal, scope, acceptance criteria, risk, eval objective

Implementer:
smallest correct code change

Verifier:
tests, schema, evidence preservation, regression comparison

Red-team:
shortcut behavior, schema-only pass, citation drift, fixture overfitting, injection

Compliance reviewer:
biosafety, IP, source licensing, auditability, privacy

Release controller:
gap classification, skipped checks, merge readiness, rollback path

- Multi-agent는 기본값이 아니다. 병렬 독립 검토, specialist handoff, or unknown subtasks가 있을 때만 쓴다.
- 반복 실패는 PR comment로 끝내지 말고 regression test, eval row, AGENTS.md rule로 전환한다.
- happy-path 통과만으로 merge하지 않는다. adversarial/edge/source coverage를 확인한다.

13. Codex Development Loop Upgrade

Codex 프롬프트는 “해썹”가 아니라 agent loop를 안정화하는 control message여야 한다. repo context, AGENTS.md, permissions, sandbox, skills, env, tests, evals, tracing, report format을 명시한다.

Codex Execution Rule:

1. Inspect AGENTS.md, README, relevant modules, existing tests first.
2. Define acceptance criteria before editing.
3. Choose simplest viable pattern.
4. Classify gaps: blocking, non-blocking, deferred-with-rationale.
5. Make smallest safe change. Avoid unrelated refactors.
6. Preserve provenance fields and JSON/schema contracts.
7. Run targeted tests first; broader tests only when shared contracts changed.
8. Add eval/regression cases for changed RAG/agent/compliance behavior.
9. Report trace, guardrail, sensitive-data, cost/latency impact.
10. Do not merge or advance stage without explicit approval and green gates.

14. Copy-ready Master Codex Prompt

Goal:

<exact behavior and success criteria>

Context:

Asperitas source-grounded, auditable, compliance-aware RAG/agent infrastructure.
Use repo patterns and AGENTS.md. Do not build a generic chatbot.

Scope:

In scope: <files/modules/features>

Out of scope: unrelated refactors, broad provider migration, undocumented behavior changes.

Why:

Explain the value for scalability, moat, biosafety/compliance, workforce leverage, or business value.

Simplest Viable Pattern:

Choose deterministic code / single LLM call / workflow / agent / multi-agent.
Justify any added complexity.

Architecture Decision:

Responses API / OpenAI Agents SDK / LangGraph / plain Python / other.
Explain why simpler options are insufficient.

State & Provenance Contract:

Preserve source IDs, citation keys, evidence spans, claim IDs, schema versions,
compliance tags, diagnostics, trace_id, eval_profile, residual uncertainty.

Tool / MCP / ACI:

Define tool schemas, examples, edge cases, permissions, side effects, timeouts,
retries, audit logs, idempotency, dry-run, prompt-injection and data-exfiltration checks.

Guardrails / HITL:

Input guardrails:

Tool guardrails:

Output guardrails:

Human approval points:

Tripwire behavior:

Implementation:

Make the smallest safe change. Reuse existing repo patterns. Avoid unrelated refactors.

Tracing:

Define workflow_name, group_id/thread_id, metadata, expected spans, sensitive-data policy.

Eval:

Define objective, dataset/fixtures, graders, pass threshold, failure taxonomy, regression scope.

Verification:

Run targeted tests first. Run broader tests only if shared contracts changed.
Report skipped checks with rationale.

PR Report:

files changed, behavior changed, tests/evals, trace coverage, guardrail impact,
sensitive-data handling, cost/latency, residual risks, reusable lesson.

15. PR Report Template

```
PR Summary:
- Goal:
- Behavior changed:
- Files changed:
- Simplest viable pattern used:
- Complexity justification:
- State/provenance impact:
- Tool/MCP impact:
- Guardrail/HITL impact:
- Trace coverage:
- Sensitive-data handling:
- Eval objective:
- Dataset/fixtures used:
- Graders/checks:
- Test commands and results:
- Skipped checks + rationale:
- Latency/cost/token impact:
- Failure taxonomy updates:
- Regression tests/eval rows added:
- Residual risks:
- Reusable lesson / AGENTS.md candidate:
- Merge readiness: ready / blocked / deferred-with-rationale
```

16. GPT 답변 성능 향상용 Project Source 지침

이 PDF를 ChatGPT Project Source로 첨부할 때, 아래 내용을 프로젝트 instructions 또는 prompt constitution 위에 얹으면 답변 품질이 안정적으로 올라간다.

```
Project Answer Quality Rule:
Every Asperitas answer should be:
- outcome-first
- source-grounded
- audit-ready
- compliance-aware
- token-efficient
- MVP-gated

Default answer structure:
1. Bottom line
2. Evidence/source basis or uncertainty boundary
3. Actionable recommendation
4. Risks/gaps
5. Verification or next PR prompt when relevant

Do not:
- overclaim production state
- invent source ingestion/legal review/vector DB/KG/eval completion
- remove provenance fields to save tokens
- optimize demo quality while weakening auditability or compliance
- treat benchmark doctrine as Asperitas implementation fact
```

17. Repository Integration Plan

이 문서를 실제 repo 규칙으로 고정하려면 PDF 첨부만으로 끝내지 말고 AGENTS.md, README, docs/dev_process.md, eval registry, tracing conventions에 반영해야 한다.

Repo artifact	Required update
AGENTS.md	Agent Development Control Plane 요약, non-negotiables, Codex prompt skeleton, stop rules
README.md	project purpose, source-grounded RAG scope, not-a-generic-chatbot boundary
docs/dev_process.md	architecture ladder, PR gates, gap closure gate, testing budget
docs/evals.md	eval objective, dataset schema, grader registry, failure taxonomy, regression policy
docs/tracing.md	workflow_name convention, group_id/thread_id, metadata, spans, sensitive data
docs/guardrails.md	input/output/tool guardrails, tripwires, HITL approvals
tests/eval_fixtures/	golden queries, adversarial cases, compliance cases, citation drift cases

Repo artifact	Required update
source_registry	license/status/version/compliance tag governance

Appendix A. Source Appendix

아래 소스들은 이 문서의 외부 benchmark layer다. 공식 문서/연구/프레임워크 문서 위주이며, 각각은 Asperitas 내부 구현 완료의 증거가 아니라 개발 품질 향상을 위한 운영 기준이다.

ID	Source	URL	Used for
S1	OpenAI API Platform	https://openai.com/ko-KR/api/	Agent platform, grounding, evals, enterprise controls
S2	OpenAI Business	https://openai.com/ko-KR/business/	company-wide AI OS, admin/security/ROI framing
S3	OpenAI Release Notes	https://openai.com/products/release-notes/	release-note-aware development, memory/plugin/security changes
S4	OpenAI Academy	https://academy.openai.com/	AI foundations, applied workflows, agents and workflow reuse
S5	OpenAI Research - Multi-agent	https://openai.com/ko-KR/research/index/?tags=multi-agent	self-play, emergent tool use, multi-agent failure/exploit lessons
S6	OpenAI Agents SDK Guide	https://developers.openai.com/api/docs/guides/agents	Responses API vs Agents SDK, specialist contracts, state, tools
S7	OpenAI Agents SDK Tracing	https://openai.github.io/openai-agents-python/tracing/	trace/span observability, sensitive data, long-running workers
S8	OpenAI Agents SDK Guardrails	https://openai.github.io/openai-agents-python/guardrails/	input/output/tool guardrails and tripwires
S9	OpenAI Evals Guide	https://developers.openai.com/api/docs/guides/evals	BDD-like evals, datasets, deprecation note, iteration
S10	OpenAI Graders	https://developers.openai.com/api/docs/guides/graders	string, similarity, model, Python graders
S11	OpenAI Codex Agent Loop	https://openai.com/index/unrolling-the-codex-agent-loop/	prompt, tools, AGENTS.md, permissions, sandbox, env context
S12	OpenAI Reasoning Best Practices	https://developers.openai.com/api/docs/guides/reasoning-best-practices	reasoning model prompt and model routing practices
S13	Anthropic Building Effective Agents	https://www.anthropic.com/engineering/building-effective-agents	simple-first, workflows vs agents, ACL, patterns
S14	Anthropic Agents in Biology	https://www.anthropic.com/research/agents-in-biology	deterministic biology retrieval layer, agent-friendly bio infrastructure
S15	Anthropic Research	https://www.anthropic.com/research	alignment, interpretability, frontier red team, societal impacts
S16	LangGraph	https://www.langchain.com/langgraph	stateful graph agent orchestration
S17	LangGraph Overview	https://docs.langchain.com/oss/python/langgraph/overview	durable execution, streaming, HITL, persistence
S18	LangGraph Persistence	https://docs.langchain.com/oss/python/langgraph/persistence	checkpoints vs stores, thread state vs long-term memory
S19	LangSmith Observability	https://docs.langchain.com/langsmith/observability	traces, metadata, tool-call observability
S20	MCP Docs	https://modelcontextprotocol.io/docs/getting-started/intro	standard tool/data/workflow connector layer
S21	Google ADK	https://docs.cloud.google.com/gemini-enterprise-agent-platform/build/adk	enterprise build/debug/deploy/evaluate/scale
S22	CrewAI Docs	https://docs.crewai.com/	crews, flows, memory, guardrails
S23	Microsoft AutoGen	https://microsoft.github.io/autogen/stable/	multi-agent conversation/team framework
S24	Semantic Kernel Agent Framework	https://learn.microsoft.com/en-us/semantic-kernel/frameworks/agent/	enterprise agent abstractions and orchestration
S25	LlamaIndex Agents	https://docs.llamaindex.ai/en/stable/understanding/agent/	RAG/data-agent loops, tools, state, HITL
S26	Ragas LlamaIndex Agent Evals	https://docs.ragas.io/en/stable/howtos/integrations/llamaindex_agents/	tool call accuracy, agent goal accuracy

ID	Source	URL	Used for
S27	DeepEval RAG Evaluation	https://deepeval.com/docs/getting-started-rag	faithfulness, answer relevancy, contextual precision/recall
S28	Phoenix Docs	https://arize.com/docs/phoenix	tracing, evals, prompt experiments
S29	Langfuse Observability	https://langfuse.com/docs/observability	LLM tracing, token/cost/latency monitoring
S30	DSPy	https://dspy.ai/	metric-driven prompt/program optimization
S31	Vercel AI SDK	https://ai-sdk.dev/docs	TypeScript tools, streaming, web deployment

Appendix B. Source Attachment Checklist

When attaching this PDF as a project source:

- Put it near P0/P1 prompt constitution materials.
- Label it as external benchmark + control-plane doctrine, not implementation proof.
- Add AGENTS.md instruction: "Apply the Agent Development Control Plane for all RAG/agent/Codex work."
- Add eval/tracing/guardrail sections into dev docs.
- For every future Codex prompt, include: Goal, Scope, Why, Simplest Viable Pattern, Complexity Justification, Tool/ACI, State/schema, Guardrails/HITL, Tracing, Eval, Verification, PR Report.
- Do not use this PDF to claim that production vector DB, KG, eval suite, legal/license review, or wet-lab validation exists unless separately verified in repo and operations.

Appendix C. One-page Operating Rule

Build the smallest source-grounded system that can be evaluated, traced, audited, and safely operated. Add agents, frameworks, and autonomy only when the evals prove that simpler patterns are insufficient.

Default Asperitas Agent Development Sequence:

```
Goal
→ Context
→ Scope
→ Why
→ Simplest viable pattern
→ Complexity justification
→ Tool/ACI contract
→ State/provenance schema
→ Guardrails/HITL
→ Implementation
→ Tests/evals
→ Tracing/observability
→ Red-team/compliance
→ Cost/latency
→ Reuse
→ PR report
```